ORGNZ-02: Understanding Grail Buckets

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 2 of 10 | **Created:** January 2026 | **Last Updated:** 11/03/2026

Overview

Buckets are logical storage containers in Dynatrace Grail where records are stored. Think of a bucket as a folder in a filesystem. Use buckets to group telemetry data that naturally belongs together, such as data from the same region, environment, or with the same sensitivity classification.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:buckets:read`, `storage:logs:read`
Knowledge	Completed ORGNZ-01 (Introduction)
Data	At least 1 hour of log data across default buckets

Table of Contents

1. What Are Buckets?
2. Built-in Grail Buckets
3. Supported Data Types (Tables)

4. System Tables
5. Bucket Limits
6. Query Constraints
7. Default Buckets
8. Querying Bucket Information
9. Querying Data from Buckets
10. Bucket Characteristics
11. Bucket Naming Rules
12. When to Create Custom Buckets

Bucket Administration Permissions

Permission	Purpose
`storage:bucket-definitions:read`	View bucket configurations
`storage:bucket-definitions:write`	Create and modify buckets
`storage:bucket-definitions:delete`	Remove buckets
`storage:bucket-definitions:truncate`	Truncate bucket data

Learning Objectives

By the end of this notebook, you will:
- Understand bucket fundamentals and supported data types
- Know bucket limits and query constraints
- Query bucket information using DQL
- Recognize when custom buckets are needed

What Are Buckets?

Buckets organize data to meet performance, compliance, and retention requirements:

Function	Description
Data Grouping	Organize large datasets into meaningful segments
Streamlined Organization	Records that should be handled together are stored together
Query Performance	Reduced queried dataset improves query speed
Retention Management	Each bucket has its own retention policy
Access Control	Bucket-level IAM policies for team isolation
Retention Cost	Storage cost follows each bucket's retention; attribute cost to teams with `dt.cost.*` fields (ORGNZ-03)

Built-in Grail Buckets

Dynatrace provides a set of predefined built-in buckets that cannot be modified. Use this DQL to discover them all:

```
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
```

Bucket Name	Table	Default Retention
`default_logs`	logs	35 days
`default_metrics`	metrics	15 months
`default_spans`	spans	10 days
`default_events`	events	35 days
`default_davis_events`	events	462 days (~15 months) — Davis events and problems
`default_bizevents`	bizevents	35 days
`default_securityevents`	security.events	1 year
`default_securityevents_builtin`	security.events	3 years

Bucket Name	Table	Default Retention
`default_user_events`	user.events	35 days
`default_user_sessions`	user.sessions	35 days
`default_mobile_user_replays`	user.replays	35 days
`default_web_user_replays`	user.replays	35 days
`default_synthetic_events`	events	35 days
`default_synthetic_user_events`	user.events	35 days
`default_synthetic_user_sessions`	user.sessions	35 days
`default_synthetic_detailed_events`	events	35 days
`default_database_monitoring`	logs	35 days — logs from Dynatrace database extensions
`default_application_snapshots`	application.snapshots	10 days
`dt_system_events`	dt.system.events	~13 months (400 days on the validation tenant)

The built-in set varies by tenant and grows as Dynatrace adds capabilities — the query above is authoritative for your environment. (Checked 09/24/2026: the validation tenant also carries `default_security_events`, `default_security` and several other `default_*` event buckets not listed here.)

> **Note:** The `default_securityevents` table was recently migrated to a new Grail security events schema. If you use security events, follow the Grail security table migration guide to complete any required actions.
>
> **Forthcoming — SaaS 1.353 (planned December 2026), announced in the SaaS 1.350 pre-release notes:** *"With Dynatrace SaaS version 1.353, planned for December 2026, security events become fully configurable in OpenPipeline, and security event storage consolidates into a single Grail bucket with a new default retention period." "All events will flow into the new default_security bucket automatically."* Act before it reaches your tenant if you query security events by bucket name — *"Notebooks, dashboards, and workflows filtering on default_securityevents or default_securityevents_builtin need updating."* — or if *"You*

> *rely on retention beyond 35 days."* Until SaaS 1.353 reaches your tenant, the security-event buckets and retention listed above remain current.
>
> **Sources:** What's new in Dynatrace SaaS 1.350 (DT docs) — pre-release notes, rollout planned from 11/10/2026.
>
> **Extended retention:** You can extend bucket retention beyond the defaults by joining the Dynatrace preview program. See Preview releases for details.



Supported Data Types (Tables)

Each bucket is associated with exactly **one data type**. You cannot mix data types in the same bucket.

Custom buckets can be created for: logs, events, security events, bizevents, spans, user events and user sessions — not for metrics.

> **Sources:** Use Grail buckets to partition data (DT docs) — *"Buckets are always associated with a specific record type, such as logs, events, spans, user events, or user sessions"*.



> **Important:** Metrics do not support custom buckets — all metric data routes to `default_metrics`.
>
> **SaaS 1.349 (staged tenant rollout from 10/27/2026):** *"Metrics now supports custom buckets."* The notes add: *"You can optionally route subsets of metrics into custom buckets using configuration rules"* (What's new in Dynatrace SaaS 1.349 (DT docs)). Verify it has reached your tenant; until then the current guidance remains the working path.

System Tables

In addition to regular data tables, Grail exposes **system tables** — virtual tables representing Grail metadata, not stored in any bucket:

System Table	Contents
`dt.system.buckets`	Bucket inventory (name, retention, size, table type)
`dt.system.data_objects`	Schema and data object metadata
`dt.system.files`	Lookup files uploaded for DQL enrichment
`dt.system.events`	Audit events (bucket changes, platform actions)

System tables are queried with `fetch dt.system.*` — they never need a `bucket:` parameter.

Bucket Limits

Environment Limits

Limit	Value	Notes
Maximum custom buckets per environment	250 (SaaS 1.346+); 80 before	Increase on request: +1 per 10 GB daily ingest; above 1,000 reviewed by Dynatrace support
Typical capacity	Up to ~2.5 TB/day per tenant with the 250 default	*"This typically satisfies raw ingestion volumes up to 2.5TB a day per tenant"*

> **Docs differ (read 10/14/2026).** The 250 default follows *How to organize your data stored in Grail* (updated 09/22/2026) — *"The default limit per environment is 250 custom buckets."* — and the SaaS 1.346 release note. *Use Grail buckets to partition data* now agrees — *"you can configure up to 250 custom buckets by default"* (re-read 10/25/2026; its date still reads 06/17/2026). The upgrade *Best practices for a logs bucket strategy* page still refers to *"the 80-bucket limit per environment"*, and *Best practices for data partitioning* (both published 08/24/2026) still says *"The default bucket limit per environment is 80 buckets, which is typically sufficient for up to 5 TB/day per table."* The two capacity figures are not reconciled; size from your own ingest volume.

Bucket Size Guidelines

Metric	Value	Source
Target ingest per log bucket	1–3 TB/day	*"Design your bucket strategy so that each bucket receives between 1 TB and 3 TB of log data per day."* At the default 500 GB scan this is ~4–12 hours of logs per query
Dedicated bucket per application	Above 250 GB/day	*"Create a dedicated bucket for any application ingesting more than 250 GB/day."* Applications below that can share a bucket
Split point, other tables	Above 2 TB/day	*"We recommend splitting up buckets that receive more than 2 TB of daily ingest."*

> **Docs differ (read 10/14/2026).** *Use Grail buckets to partition data* (updated 06/17/2026) gives the 2 TB split point for all buckets. *Best practices for a logs bucket strategy* (published 08/24/2026) is newer and specific to logs and gives 1–3 TB/day, so this table follows it for logs; for other tables the 2 TB figure is the only one stated.
>
> **Sources:** Best practices for a logs bucket strategy (DT docs), Best practices for data partitioning (DT docs), Use Grail buckets to partition data (DT docs), How to organize your data stored in Grail (DT docs).

Retention Limits

Limit	Value
Minimum retention	**1 day** for logs, events, and bizevents; **10 days for spans**
Maximum retention	3,657 days (~10 years + 1 week)

Query Constraints

Understanding query limits is critical for bucket planning:

Query Limits

Limit	Value	Impact
Default `fetch` scan limit	500 GB	Default value of the `scanLimitGBytes` parameter — **not** a platform cap. Override per query; `scanLimitGBytes:-1` scans the whole time range
Default records returned	1,000	A default, not a cap: raise `maxResultRecords` on a Query API request, or aggregate for larger datasets
Default result payload	1 MB	A default (`maxResultBytes`); larger results stop at about this size unless it is raised

> **Sources:** Best practices for a logs bucket strategy (DT docs) — lists *"500 GB scanned per query"*, *"1,000 records returned per result"* and *"1 MB result payload"* as *"default query limits"*, Grail storage query SDK (Dynatrace Developer) — `maxResultRecords`: *"The maximum number of query result records to be returned. If this limit is exceeded, a warning notification will be included in the response."*

Queryable Window by Ingest Volume

With the default 500 GB `fetch` scan limit, a single unmodified query reaches back roughly:

Daily Ingest	Queryable Window
500 GB/day	~24 hours
1 TB/day	~12 hours
3 TB/day	~4 hours
10 TB/day	~1 hour

> **Implication**: High-volume buckets may only allow querying recent data. Use aggregations, time filters, and strategic bucket partitioning to work within these limits.

Default Buckets

Dynatrace provides default buckets with varying retention:

Default Bucket	Data Type	Default Retention	Notes
`default_logs`	logs	35 days	Standard log retention
`default_metrics`	metrics	15 months	Extended for trend analysis
`default_spans`	spans	10 days	Short-term APM data

Default Bucket	Data Type	Default Retention	Notes
`default_events`	events	35 days	Platform events
`default_bizevents`	bizevents	35 days	Business events
`default_securityevents`	security.events	1 year	Security events
`default_securityevents_builtin`	security.events	3 years	Built-in security events

> **Note**: Query your environment's `dt.system.buckets` to verify current retention settings.

Querying Bucket Information

DQL: Built-in Bucket Discovery

Discover all built-in Grail buckets in your environment:

```
// List all built-in Grail buckets — inventory check
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
| fields name, display_name, dt.system.table, retention_days
| sort dt.system.table asc, name asc
```

DQL: Bucket Audit Events

Track who created, updated, truncated, or deleted a bucket:

```
// Audit bucket management actions — returns create, update, truncate, delete events
// Bucket changes are rare: widen to from:-365d for a full year of history
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT" and event.category == "BUCKET_MANAGEMENT"
| fields timestamp, event.type, resource, user.id, event.outcome
| sort timestamp desc
```

Querying Data from Buckets

DQL: Querying Data from Buckets

Use the `bucket:` parameter to target specific storage:

```
// Query logs from a specific bucket — use the bucket: parameter to target
storage
fetch logs, from:-1h, bucket:{"default_logs"}
| limit 10
```

```
// Query from multiple buckets simultaneously — controls scan scope
// Bucket names that don't exist are skipped with no warning — a misspelled
name looks like an empty bucket.
// Check names first: fetch dt.system.buckets | filter dt.system.table ==
"logs" | fields name
fetch logs, from:-1h, bucket:{"default_logs", "audit_logs", "security_logs"}
| limit 100
```

Bucket Characteristics

Immutability Rules

Characteristic	Mutable?	Notes
Bucket name	No	Cannot be changed after creation
Display name	Yes	Can be updated anytime
Retention period	Yes	**Caution**: Lowering deletes data
Data table type	No	Fixed at creation
Bucket class (live / historic)	No	Read-only after creation

Data Handling

Operation	Supported?	Alternative
Move data between buckets	No	Route new data via OpenPipeline

Operation	Supported?	Alternative
Selective record deletion	No	Wait for retention to expire
Bucket deletion	Yes	Deletes ALL data permanently

Bucket Naming Rules

Rule	Requirement
Name length	3–100 characters
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot use `default_*` prefix

Valid Examples

```
team_logs_30d
finance-audit-logs
prod_events_90d
app123_spans
```

Invalid Examples

```
123_logs          # Starts with number
Team_Logs         # Contains uppercase
default_custom    # Uses reserved prefix
my logs           # Contains space
```

When to Create Custom Buckets

Need	Solution
Different retention periods	Create bucket with specific retention
Cost attribution by team/LOB	Not a reason on its own — use `dt.cost.costcenter` / `dt.cost.product`; add a bucket only where that cost center also needs its own retention or access
Simple team isolation	Bucket-level IAM policies
Regulatory compliance	Dedicated buckets for compliance data
Query performance	Isolate high-volume data sources
Debug/verbose logs	Short-retention bucket to reduce costs

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-03**: Bucket Strategy and Design

References

- Use Grail buckets to partition data (DT docs)
- How to organize your data stored in Grail (DT docs)
- Best practices for a logs bucket strategy (DT docs)
- Best practices for data partitioning (DT docs)
- Data Retention
- DQL Reference

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.